

Genestrip: Efficient read classification, filtering and k -mer counting for selected groups of species

Daniel Pfeifer (daniel.pfeifer@hs-heilbronn.de)

January 2025

The consumption of computational resources is a significant burden in modern metagenomic analysis even when applying (unique) k -mer-counting. Genestrip overcomes this problem by focusing on small but freely selectable groups of species. Regarding the selected species, it produces results comparable to Kraken 1 and KrakenUniq but at a fraction of their required memory resources. Thus, mass data analysis may be performed even on small-sized PCs or servers. Moreover, Genestrip can be used to extremely reduce the size of sequence files for downstream steps without loss of relevant data regarding the selected species. We assess the usefulness, correctness and performance of Genestrip in the context of tick surveillance, ?? and

1. Introduction

With the availability of low cost and small-scale sequencing devices, metagenomic analysis will likely replace many established technologies for the detection and surveillance of specific groups of microorganisms. Related applications may affect various areas such as farming, ecology, food production and medicine. While sequencing devices have become ready for ubiquitous use, software tools for metagenomics still tend to be resource intensive and hard to use for end users.

We present Genestrip, a new metagenomic tool based on k -mer matching and filtering that addresses the need for low resource consumption and ease of use. Genestrip is based on Java and can be run on all modern operating systems. ... competitive performance ...

2. Background

To analyze large sets of reads, researchers use software tools based on k -mer matching and counting such as Kraken 1 (Wood and Salzberg 2014), Kraken 2 (Wood, Lu, and Langmead 2019) or KrakenUniq (Breitwieser, Baker, and Salzberg 2018). With regard to pathogen detection KrakenUniq is particularly suited because of its very low false positive rate but also because its efficiency, precision and sensitivity (Breitwieser, Baker, and Salzberg 2018).

In order to avoid false positive classifications of reads or mismatches of k -mers, all of these tools resort to very large databases, containing millions of encoded k -mers along with their respective taxons. A database is usually loaded entirely into main memory and consumes tens of gigabytes of space¹. This requirement mandates specialized and expensive compute servers with very large main memory.

In recognition of this problem, KrakenUniq has been extended for database processing in chunks (Pockrandt, Zimin, and Salzberg 2022). However, ...

If only a smaller group of a few 100 species/strains is under consideration, then it seems promising to simply *generate a smaller database that contains just the k -mers from respective genomes. However, this very likely leads to many false positives* even when using KrakenUniq. The reason for this is that due to the evolutionary relationship of organisms, *a large fraction of k -mers that belong to the genome of a species are unspecific* in a sense that they can as well be found in the genome other species not considered for analysis. Thus, when using too small a database, a read may be falsely assigned to a species because it consists of k -mers that *also* belong to another species' genome that was not included in the database.

In order to enable a smaller database with no false positives emerging from this problem, Genestrip employs a *two-phase* database generation process: During the *entry phase*,

¹See <https://benlangmead.github.io/aws-indexes/k2> for related Databases

all k -mers of genomes from selected species are entered into the database on the basis of the RefSeq (O’Leary et al. 2016) and optionally GenBank (Sayers et al. 2019). During the *update phase*, all stored k -mers from the database are checked against the genomes of other species from the RefSeq. If a stored k -mer also occurs in another species which is not part of the database, the k -mer’s tax id gets updated in the database and the least common ancestor’s taxonomic identifier is entered instead. This prevents k -mers from being associated with one of selected species even though they are unspecific.

Based on this approach we achieve very low memory footprints during analysis and a competitive k -mer matching and read classification performance. The actual k -mer counting results are similar to those of Kraken 1 (Wood and Salzberg 2014) or KrakenUniq (Breitwieser, Baker, and Salzberg 2018) with regard to the selected species. Much as KrakenUniq, Genestrip supports unique k -mer counting with exact counts but with very low memory overhead.

3. Methods

3.1. System Structure

Genestrip builds on the specification of selected groups of species whose genomes will be represented accurately and exclusively in a resulting database. The desired genres, species and strains must be set via a configuration file. Users may choose organisms by listing their taxonomic identifiers in that file. If higher ranked taxonomic identifiers are entered, then *all* of the the descending organisms will be included and k -mers of their genomes will be added to the database accordingly.

Genestrip generates a database for a selected group of species mainly via the reference genomes from the RefSeq (O’Leary et al. 2016). To better handle the complexity of the generation process, we employ a goal-oriented approach in similarity to Make (Feldman 1979). On access, each goal returns a file or an object in main memory. A goal is only executed if its result is not yet available. Goal execution may involve the access of other goals and which turn, may trigger their execution. Respective goal dependencies are explicitly specified in terms of a directed acyclic graph.

Figure 1 depicts Genestrip’s entire goal graph: Several goals (prefixed with `d:`) revolve around downloading files from the RefSeq and other data from the NCBI. Several object goals (prefixed with `o:`) load files and represent them as data structures in main memory for further processing. Most file goals (prefixed with `f:`) write computational results to files. Boxes with edged corners depict goals that can be executed directly via the command line. E.g., the goals `match` and `matchlr` are for fastq file analysis but they would trigger a respective database generation if the database file was missing.

3.2. Database and Filter Generation

Genestrip stores a database primarily as a large array S of sorted long values. Each long value encodes a k -mer from a reference genome of a selected species. The default for k is 31 but other values are possible. A second array of the same size encodes taxonomy identifiers for each k -mer in S . During fastq file analysis, lookups in S are performed via binary search.

An additional Bloom filter (Bloom 1970) also contains all stored k -mers. It is used by default at analysis time to check whether a certain k -mer is stored in S . A negative results avoids a more expensive binary search in S and therefore saves overall execution cost.

Moreover, the parts of the NCBI taxonomy tree (Schoch et al. 2020) required at analysis time are included in the database. The taxonomy extract comprises entries for the selected species and all of its ancestors.

Table 2 describes the most relevant goals for database generation in their typical execution order.

Besides k -mer matching and read classification, Genestrip also supports filtering of fastq files via a Bloom filter in similarity to (Chu et al. 2014). When generated via the goal `index` the Bloom filter gets stored in separate file. The goal `filter` loads the filter and processes fastq files accordingly. The standard configuration for the filtering process rewrites a read to the output if the read contains at least one k -mer that is recognized by the filter and thus may belong to a selected species.

3.3. k -mer Matching and Counting

3.4. Statistical considerations for k -mer counts

Let $k_{t,f,d}$ be the number of k -mers for taxonomy identifier t found in fastq file f with respect to database d and $u_{t,f,d}$ be the corresponding number of unique k -mers. Similarly $u_{t,d}$ is the number of (unique) k -mers for t stored in d , k_f the total number of k -mers in f , u_d the total number of (unique) k -mers stored in d and so on.

Let F be the set of analyzed fastq-files. In order to compare $k_{t,f,d}$ between different files in $f \in F$ and potentially different databases d or taxonomy identifiers t , we need to account for the a-priori bias caused by different file sizes k_f and different numbers of stored k -mers $u_{t,d}$ of different taxonomy identifiers t and different databases d .

We therefore consider the a-priori probability $p_k(t, f | d)$ which denotes the probability of t to occur in f given that (unique) k -mers of t are stored in d . As this is an a-priori probability, we must assume independency between t and f . As f does not depend on d this gives:

$$p_k(t, f | d) = p(f | d) \cdot p(t | d) = p(f) \cdot p(t | d) \approx k_f / \sum_{g \in F} k_g \cdot u_{t,d} / u_d$$

We suggest a new measure – *the unbiased k -mer count* $k'_{t,f,d}$ that accounts for the a-priori bias:

$$k'_{t,f,d} := c_F \cdot k_{t,f,d} / k_f \cdot u_d / u_{t,d} \approx k_{t,f,d} / p_k(t, f | d)$$

where c_F captures the normalizing factor for F . As we don't depend on values in the range of $[0, 1]$, we can simply set $c_F := 1$.

Given $k_{t,f,d}$, the a-priori probability $p_u(t | k_{t,f,d})$ of a unique k -mer for t from d to be found *at least once* in f is:

$$p_u(t | k_{t,f,d}) = 1 - (1 - 1/u_{t,d})^{k_{t,f,d}}$$

Therefore, the corresponding *expected number of unique k -mers* E_q is:

$$E_q(k_{t,f,d}, u_{t,d}) = u_{t,d} \cdot p_u(t | k_{t,f,d})$$

E_q is very useful as we can compare this expectation with the actual count $u_{t,f,d}$. If these quantities divert too much from each other, then the consistency of the counting results for t in f should be questioned. If, on the other hand, these quantities are close, then this supports the consistency of the counting results. This justifies the following ratio as a consistency score, which ideally is close to 1:

$$r(k_{t,f,d}, u_{t,f,d}) := u_{t,f,d} / E_q(k_{t,f,d}, u_{t,d})$$

A more mathematically sound approach is to consider the probability distribution $q(u_{t,f,d} | k_{t,f,d})$, i.e. the probability to obtain exactly $u_{t,f,d}$ unique k -mers for a given (non-unique) k -mer count of $k_{t,f,d}$ with sampling with replacement as the underlying random process. Interestingly, this distribution is non-trivial and has been investigated in Mendelson et al. (2016). The authors have shown that the distribution can be approximated by a normal distribution under most practically relevant conditions. Moreover, they confirm the expectation E_q from above and give a formula for the variance σ_q^2 which in our case depends on $k_{t,f,d}$ and $u_{t,d}$.

So, as a second, alternative consistency score we set

$$r'(k_{t,f,d}, u_{t,f,d}) := f(u_{t,f,d}) / f(E_q),$$

where f is the density of the normal distribution $\mathcal{N}(E_q, \sigma_q^2)$. This relative density r' ranges between 0 and 1 where 1 is best and implies $u_{t,f,d} = E_q$.

4. Results

4.1. Example application: Pathogen surveillance in ticks

4.1.1. Fastq files

Total number of fastq files:

Average g-zipped fastq file size:

Maximum g-zipped fastq file size:

Total g-zipped file size:

Total number of reads:

Total number of k -mers:

4.1.2. Selected pathogens

Tick-borne disease geni covered:

Number of taxons for species/strain covered:

Number of fasta files:

Total g-zipped fasta file size: Total number of k -mers:

4.2. Correctness of k -mer counting

4.3. Correctness and efficiency of filtering

Average/max/min reduction in reads per fastq file:

4.4. Database statistics

Stored k number of k -mers.

Ratio stored / per total k -kmers per genus.

g-zipped database size.

4.5. Performance

Server configuration:

Database generation time:

Main memory consumption:

Database loading time:

Read processing speed:

Total processing time for all fastq file from Section ??.

Number of threads: 9

Comparing numbers for KrakenUniq...

4.6. Average specific k -mer contig length (askcol) as a new feature for analysis

4.7. Finding Species-specific k -mers

Generating fastq file with from fasta file with one read per k -mer as given by sliding window in genome sequence of fasta file.

Run fastq file against Kraken 1/KrakenUniq to obtain taxon of each k -mer. Keep k -mers with taxon belonging to taxon set of chosen group of species. Store such k -mers along with its taxon in Genestrip database.

5. Discussion

References

- Bloom, Burton H. 1970. "Space/time trade-offs in hash coding with allowable errors." *Commun. ACM* 13 (7): 422–426. <https://doi.org/10.1145/362686.362692>.
- Breitwieser, F. P., D. N. Baker, and S. L. Salzberg. 2018. "KrakenUniq: confident and fast metagenomics classification using unique k -mer counts." *Genome Biology* 19 (1): 198. <https://doi.org/10.1186/s13059-018-1568-0>.
- Chu, Justin, Sara Sadeghi, Anthony Raymond, Shaun D. Jackman, Ka Ming Nip, Richard Mar, Hamid Mohamadi, Yaron S. Butterfield, A. Gordon Robertson, and Inanç Birol. 2014. "BioBloom tools: fast, accurate and memory-efficient host species sequence screening using bloom filters." *Bioinformatics* 30 (23): 3402–3404. <https://doi.org/10.1093/bioinformatics/btu558>.
- Feldman, Stuart I. 1979. "Make—a program for maintaining computer programs." *Software: Practice and experience* 9 (4): 255–265.
- Mendelson, Alex F., Maria A. Zuluaga, Brian F. Hutton, and Sébastien Ourselin. 2016. "What is the distribution of the number of unique original items in a bootstrap sample?". <https://arxiv.org/abs/1602.05822>.
- O'Leary, Nuala A, Mathew W Wright, J Rodney Brister, Stacy Ciufu, Diana Haddad, Rich McVeigh, Bhanu Rajput, Barbara Robbertse, Brian Smith-White, Danso Ako-Adjei, Alexander Astashyn, Azat Badretdin, Yiming Bao, Olga Blinkova, Vyacheslav Brover, Vyacheslav Chetvernin, Jinna Choi, Eric Cox, Olga Ermolaeva, Catherine M Farrell, Tamara Goldfarb, Tripti Gupta, Daniel Haft, Eneida Hatcher, Wrutko Hlavina, Vinita S Joardar, Vamsi K Kodali, Wenjun Li, Donna Maglott, Patrick Masterson, Kelly M McGarvey, Michael R Murphy, Kathleen O'Neill, Shashikant Pujar, Sanjida H Rangwala, Daniel Rausch, Lillian D Riddick, Conrad Schoch, Andrei Shkeda, Susan S Storz, Hanzhen Sun, Francoise Thibaud-Nissen, Igor Tolstoy, Raymond E

- Tully, Anjana R Vatsan, Craig Wallin, David Webb, Wendy Wu, Melissa J Landrum, Avi Kimchi, Tatiana Tatusova, Michael DiCuccio, Paul Kitts, Terence D Murphy, and Kim D Pruitt. 2016. "Reference sequence (RefSeq) database at NCBI: current status, taxonomic expansion, and functional annotation." *Nucleic Acids Res* 44 (D1): D733–45.
- Pockrandt, Christopher, Aleksey V Zimin, and Steven L Salzberg. 2022. "Metagenomic classification with KrakenUniq on low-memory computers.." *J Open Source Softw* 7 (80).
- Sayers, Eric W, Mark Cavanaugh, Karen Clark, James Ostell, Kim D Pruitt, and Ilene Karsch-Mizrachi. 2019. "GenBank." *Nucleic Acids Research* 48 (D1): D84–D86. <https://doi.org/10.1093/nar/gkz956>.
- Schoch, Conrad L, Stacy Ciufu, Mikhail Domrachev, Carol L Hotton, Sivakumar Kannan, Rogneda Khovanskaya, Detlef Leipe, Richard Mcveigh, Kathleen O'Neill, Barbara Robbertse, Shobha Sharma, Vladimir Soussov, John P Sullivan, Lu Sun, Seán Turner, and Ilene Karsch-Mizrachi. 2020. "NCBI Taxonomy: a comprehensive update on curation, resources and tools.." *Database (Oxford)* 2020.
- Wood, Derrick E., Jennifer Lu, and Ben Langmead. 2019. "Improved metagenomic analysis with Kraken 2." *Genome Biology* 20 (1): 257. <https://doi.org/10.1186/s13059-019-1891-0>.
- Wood, Derrick E., and Steven L. Salzberg. 2014. "Kraken: ultrafast metagenomic sequence classification using exact alignments." *Genome Biology* 15 (3): R46. <https://doi.org/10.1186/gb-2014-15-3-r46>.

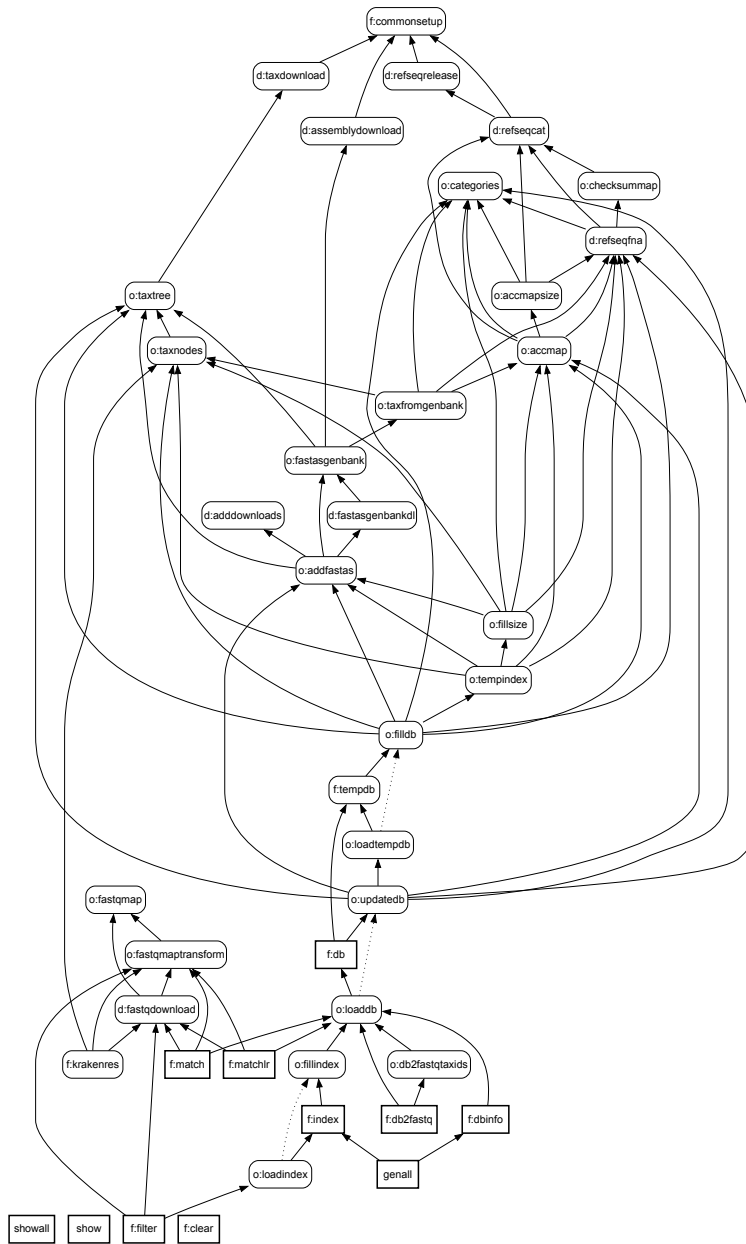


FIGURE 1. Dependency Graph of Goals in Genestrip

Goal Name	Description
fillsize	Determines the maximum size of the database in number of k -mers by reading over all reference genome files and including genomes of selected species. It is a very conservative estimate as duplicates are double counted.
tempindex	Creates a Bloom filter filled with all k -mers of genomes of selected species. The Bloom filter's size is initially set via the result from <code>fillsize</code> . As the Bloom filter avoids double counting, its final number of entries can serve as a more exact basis for the actual database size.
fillldb	Initially populates the database with all k -mers of genomes from selected species. The appropriate size of the database is a close but conservative estimate based on <code>tempindex</code> . Duplicate k -mer entries are avoided via a Bloom filter that tracks the already stored k -mers. Finally, the stored k -mers are sorted in order to enable binary search at analysis time and for the goal <code>updatedb</code> from below.
updatedb	Updates the populated database to account for common k -mers between species. <i>All</i> genomes from the RefSeq belonging to a set of pre-configurable taxonomic kingdoms will be considered. This step ensures that a k -mer from a selected species obtains the correct taxonomic identifier as the k -mer may also occur in the genome of another species (which might be represented in database or not). Concerning the finished database, the respective k -mer will be associated with least common taxonomic ancestor of all species whose genome(s) contain(s) the k -mer.

FIGURE 2. Description of Genestrip's most Relevant Goals for Database Generation.



FIGURE 3. k -mer counts of Generstrip vs. Kraken 1 for the group of taxons from section ???. Every point represents a result for one taxon and for one fastq file from ???.



FIGURE 4. Unique k -mer counts of Generstrip vs. KrakenUniq for the group of taxons from section ???. Every point represents the result for one taxon and for one fastq file from ???.



FIGURE 5. k -mer counts of Kraken 1 for filtered vs. unfiltered fastq files for the group of taxons from section ???. Every point represents a result for one taxon and for one fastq file from ??.



FIGURE 6. askol vs. unique k -mer of KrakenUniq files for the group of taxons from section ???. Every point represents a result for one taxon and for one fastq file from ??.